BUAA_2025_Unit1_表达式解析与递归下降

架构设计迭代

1.1 第一次作业的架构分析

本次作业整体~~copy~~沿用了 OOPre 中"**预处理 + 词法分析 + 语法分析 + 语义计算**"的分层结构。

首先根据文法，可以提取出最基本的 token，包括：

- `+`
- `-`
- `*`
- `^`
- `(`
- `)`
- `x`
- 数字字符

因此在词法层面，只需要让 `Lexer` 识别这些最小单元，并将输入划分为 token 流即可。

在语法层面，表达式可以自然分成三层结构：

- `Expr`
- `Term`
- `Factor`

其中，`Expr` 由若干项组成，`Term` 由若干因子相乘构成，而 `Factor` 又分为三类：

- 常数因子 `Number`
- 幂函数因子 `Var`
- 表达式因子 `ExprFactor`

因此在 `Parser` 中，通过 `parseExpr()`、`parseTerm()` 和 `parseFactor()` 递归下降地进行解析，就可以将输入构造成表达式树。

1.2 输入预处理与解析

在正式解析前，我使用 `Process` 对输入进行预处理，主要包括：

- 去除空白字符
- 合并连续正负号
- 去掉指数前多余的正号，如 `^+2 -> ^2`

这样可以让后续词法和语法分析更简单。

预处理后，`Lexer` 负责把字符串转化为 token 流，例如加减号、变量、数字、括号等；`Parser` 再按照文法逐层解析，得到 `Expr`、`Term` 和 `Factor` 对象。这样整个表达式就从字符串形式变成了结构化的表达式树。

1.3 表达式展开

分析可知，本次表达式展开后的结果一定可以写成多项式形式：

$ Expr=\sum a_i x^i $

我设计了：

- `Mono`：表示单项式
- `Poly`：表示多项式

其中，`Mono` 保存系数和指数，`Poly` 负责保存多个单项式，并完成加、减、乘、幂以及合并同类项等操作。

在实现上，各个语法结点统一提供 `toPoly()` 方法：

- `Expr` 负责项之间的加减
- `Term` 负责因子之间的乘法
- `Factor` 的不同实现分别负责把常数、变量和括号表达式转成多项式

这样就完成了从“语法结构”到“代数结构”的转化。

1.4 Bug 分析

比较幸运的是，这次强测和互测中我没有出现实际 bug。

在第一次作业房间中，即使多次利用评测机进行 hack，也没有成功构造出错误点。

1.5 性能优化

本次作业中的优化，除了运算过程中的时间复杂度，还包括**输出字符串长度的优化**。

例如：

```
-3*x+3
```

更优的输出形式应为：

```
3-3*x
```

因为如果存在正项，让正项最先输出，就可以少一个前导负号。在实现时，没有必要专门进行复杂排序，只需要在输出时找到第一个正项并优先输出即可。 ~~学OO是不是会很开心~~ ~~以后的优化是不是也会如此Easy~~

`Mono` 层优化

在单项式输出时，我做了以下简化：

- 系数为 `0`，直接不输出

- 指数为 `0`，直接输出系数
- 系数为 `1` 时省略 `1`
- 系数为 `-1` 时只保留负号
- 指数为 `1` 时省略 `^1`

这些规则按优先级依次处理，可以有效减少冗余字符。

`Poly` 层优化

在多项式输出时，我主要做了以下处理：

- 若某一项为空串，则直接跳过
- 非空项之间用 `+` 连接
- 若存在正项，则尽量把正项放在最前面
- 若最后所有项都被消去，则输出 `0`

这样既保证了结果正确，也尽可能缩短了最终输出长度。

最后展示一下第一次作业中的 UML 类图和复杂度图：

class	OCavg	OCmax	WMC
expr.Expr	1.50	3	6

expr.ExprFa	1.00	1	2
expr.Numb	1.00	1	2
expr.Term	1.50	3	6
expr.Var	1.00	1	2
main.Main(	1.00	1	1
main.Proce	1.50	2	3
parser.Lexe	3.50	11	14
parser.Pars	4.00	6	16
parser.Toke	1.00	1	2
parser.Token.Typ			0
poly.Mono	1.20	2	6
poly.Poly	5.38	17	43
Total			**103**
Average	2.58	4.08	7.92

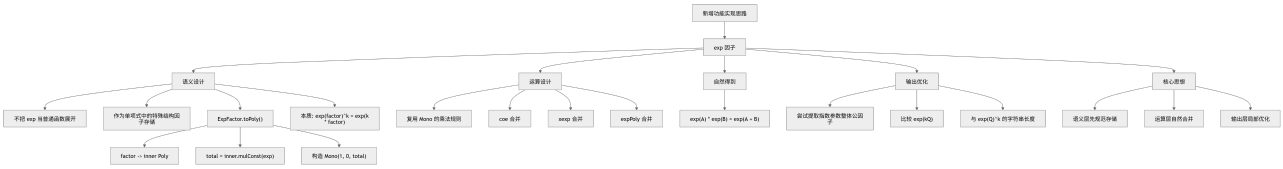
package	v(G)avg	v(G)tot
expr	1.36	19
main	1.33	4
parser	3.90	39
poly	3.85	50
Total		**112**
Average	2.80	28.00

2. 第二次作业的架构设计迭代

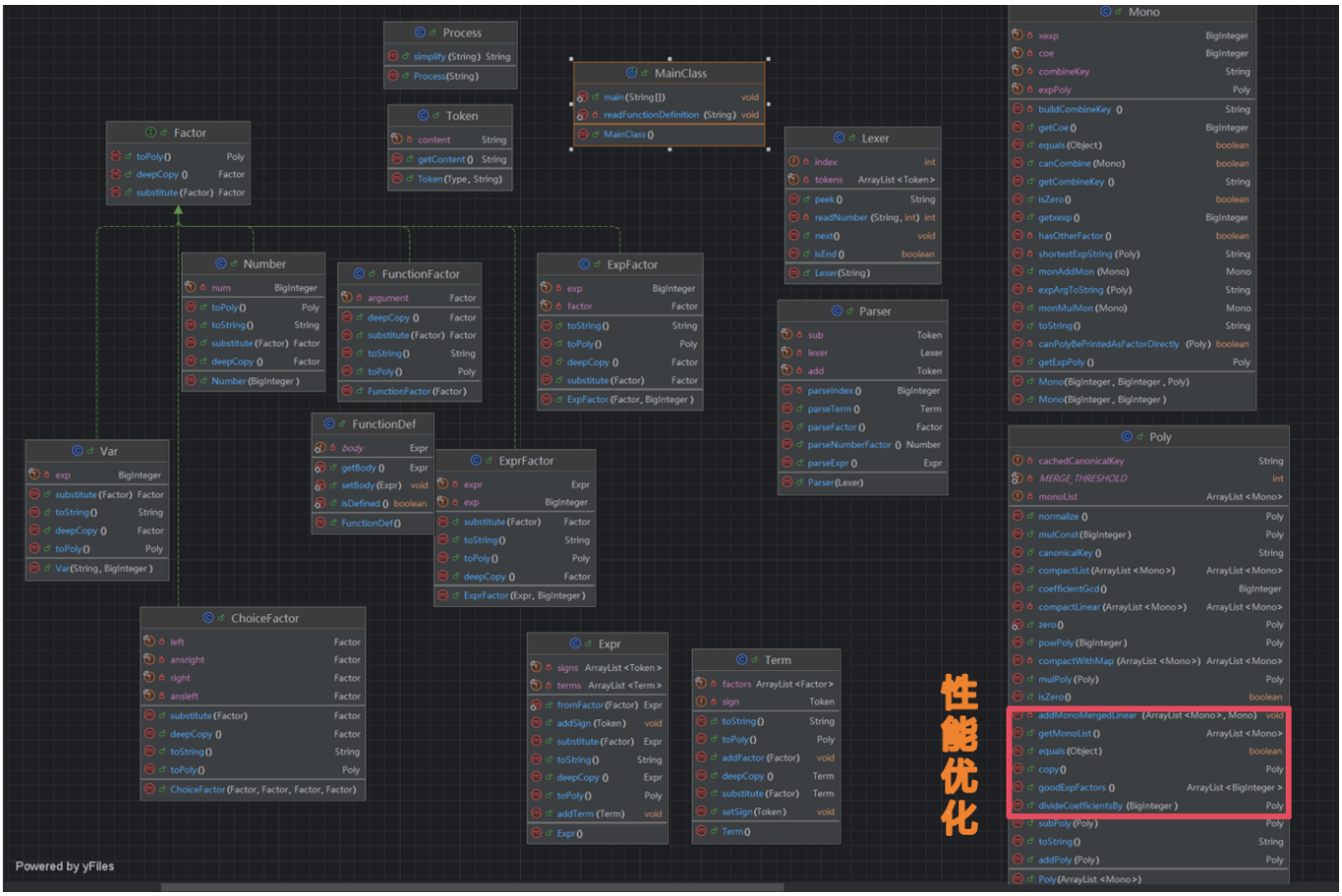
2.1 架构分析

本次新增了以下三类语言特性：

- 指数函数因子 `exp`
- 选择式因子 `[(A==B)?C:D]`
- 自定义函数 `f(x)=Expr` 的定义与调用



下面给出本次作业的 UML 类图：



2.2 新功能的实现思路

2.2.1 exp 因子的实现

本次作业中，`exp` 的处理并不是将其视为普通函数后立即展开，而是将其看作**单项式中的一种结构性因子**来保存。这样的设计使得后续乘法合并与字符串优化都更自然。

在实现中，`ExpFactor.toPoly()` 会先将内部参数 `factor` 转化为规范化后的多项式 `inner`，再结合外层幂次构造出对应的指数参数。也就是说，并不会试图把 `exp(...)` 进一步展开成普通多项式，而是将其作为 `Mono` 的特殊字段进行维护。

这样设计的直接好处在于：对于两个指数因子的乘积

$$ \exp(A)\cdot \exp(B) $$

可以在单项式乘法时自然合并为

$$ \exp(A+B) $$

即不需要额外编写复杂特判，只需在单项式乘法中分别合并：

- 系数
- 变量指数
- 指数函数参数

即可完成语义正确的合并。

进一步地，在输出阶段，我并不局限于直接打印 `exp(P)`，而是会尝试对指数参数中的系数进行提取。例如，对于

$$ \exp(kQ) $$

程序会比较输出为

$$ \exp(kQ) $$

与

$$ \exp(Q)^k $$

两种形式时的字符串长度，并选择更短的一种进行输出。

这说明 `exp` 的处理策略可以概括为：

> 将 `exp` 视为单项式的结构性组成部分；在语义层保持规范存储，在运算层通过单项式规则自然合并，在输出层再进行局部长度优化。

2.2.2 选择式因子 `[(A==B)?C:D]` 的实现

选择式因子的核心在于：程序需要判断 `A` 与 `B` 是否**数学恒等**，而不仅仅是判断它们的字符串形式或语法树结构是否相同。

在实现中，`ChoiceFactor.toPoly()` 会先将条件左右两侧分别转换为规范化后的多项式：

- `left.toPoly().normalize()`
- `right.toPoly().normalize()`

随后通过 `Poly.equals()` 判断两者是否相等。若相等，则返回 `C` 对应的结果；否则返回 `D` 对应的结果。

这里的关键点在于，`Poly.equals()` 并不是简单地逐项比较当前存储顺序，而是将多项式转换为稳定的标准表示后再进行比较。

> 两个表达式在规约到统一标准型之后，是否得到完全相同的代数对象。

> 条件判断不直接比较原始表达式，而是先将两侧规约到统一的多项式标准型，再进行语义层面的相等判断。

2.2.3 自定义函数 `f(x)=Expr` 的实现

对于自定义函数，我采用了“**定义与调用分离**”的实现策略。

在读入阶段，程序首先解析函数定义，将等号右侧表达式构造成语法树，并保存到 `FunctionDef` 中；在后续主表达式解析过程中，若遇到函数调用 `f(...)`，再从定义中取出函数体并进行展开。

需要强调的是，这里的函数展开并不是字符串替换，而是**基于语法树的结构化代入**。具体流程如下：

1. 取出函数体对应的 AST；
2. 深拷贝一份，避免污染原始定义；

3. 对拷贝后的语法树递归执行 `substitute(argument)`；
4. 将代入后的新表达式统一转换为 `Poly` 并规范化。

其中，各类语法结点都实现了 `substitute()` 方法。例如，变量因子在代入时会按指数情况分类处理：

- $x^0 \to 1$
- $x^1 \to arg$
- $x^k \to (arg)^k$

2.3 表达式优化

2.3.1 已考虑到的优化

在当前实现中，我已经考虑到以下事实：

对于两个相乘的指数因子，直接合并通常能够缩短结果长度。例如：

$$ \exp(x^2)\cdot \exp(x)\rightarrow \exp(x+x^2) $$

由于指数参数可以直接相加，因此这类合并通常是收益明确的。

同时，对于形如 $\exp(kQ)$ 的表达式，我也实现了局部的公因子提取优化，即比较

$$ \exp(kQ)\quad \text{与} \quad \exp(Q)^k $$

哪种输出形式更短，再决定最终采用哪一种。

2.3.2 未充分考虑到的情况

不过，进一步分析后可以发现，仅仅进行“指数合并”与“整体提取公因子”仍然不够，因为：

1. **将指数乘入 `exp` 内部不一定总是最优**

   例如：

   $$ \exp(2x)^2 \rightarrow \exp(4x) $$

   在这种情况下，直接将指数乘入内部更短；但对于

   $$ \exp(9x+9x^2)^2 $$

   若先整体乘入得到

   $$ \exp(18x+18x^2) $$

   未必就是最短形式，因为也可能写成

   $$ \exp(x+x^2)^{18} $$

   从而在字符串长度上更优。

2. **先拆分，再提取公因子，有时会比直接合并更优**

例如：

$$ \exp(70x^8+60x^7+50x^6+40x^5+30x^4+20x^3+10x^2+x) $$

若直接保持整体形式，其长度为 57；但若先拆成

$$ \exp(x)\cdot \exp(70x^8+60x^7+50x^6+40x^5+30x^4+20x^3+10x^2) $$

则第二部分内部会出现公因子 `10`，可进一步写为

$$ \exp(x)\cdot \exp(x^2+2x^3+3x^4+4x^5+5x^6+6x^7+7x^8)^{10} $$

最终长度反而缩短为 56。

这说明：

> `exp` 的最短输出形式，并不一定来自“完全合并”或“完全不合并”，而可能来自某种中间状态下的拆分方案。

2.3.3 问题本质与后续思考

因此，这一优化问题的本质可以表述为：

> 对于指数参数为某个多项式的 `exp` 因子，如何寻找一种拆分方式，使得在允许公因子提取的前提下，最终输出字符串长度最短？

如果对所有拆分方案进行暴力枚举，时间开销会非常大，在强测环境下难以接受。因此，本次作业中我并未继续深入实现全局最优拆分，而只是完成了局部优化策略。

这也正是我在强测中丢失性能分的主要原因之一：**尚未考虑 `exp(P)` 进一步拆分成 `exp(Q)*exp(S)` 的可能性。**

2.4 Bug 分析

2.4.1 强测中的问题

本次强测中，我没有出现功能性错误，但丢失了一部分性能分。主要原因在于：

- 仅考虑了 `exp` 因子的直接合并；
- 未进一步考虑 `exp(P)` 的拆分及其与公因子提取结合后的更优输出形式。

2.4.2 hack

在互测阶段，我成功构造出了针对部分实现的 hack 数据，例如：

```
1
f(x)=exp((exp(exp(exp(exp(exp(x))))))+x))
[(5==f(x))?1:[(1==f(x))?1:[(1==f(x))?1:[(1==f(x))?1:f([(4==1)?1:f(f(f(x)))])]]]]
```

该数据主要利用了以下几点：

- 深层嵌套 `exp` 带来的表达式膨胀；
- 选择式因子的多层嵌套判断；
- 自定义函数的递归式展开；
- 在判等与展开交织时造成较高的时间开销。

通过这组数据，我一次性成功 hack 到了三位同学的 TLE 问题。

总体而言在第二次作业的设计中

- `exp` 的设计重点在于将其作为单项式的结构性因子保存，而不是简单视为普通函数；
- 选择式因子的关键在于将“数学恒等判断”转化为“标准型相等判断”；
- 自定义函数则通过 AST 级代入避免了字符串替换带来的不稳定性。

不过，从本次作业的结果来看，我在**正确性实现**上达到了预期，但在**输出长度**方面仍有提升空间。特别是对于 `exp` 因子的拆分与组合问题，当前实现只覆盖了局部最优情况，尚未解决拆分成乘积形式的全局最优表达形式的搜索问题。这也是hw3优化时重点深入的方向。

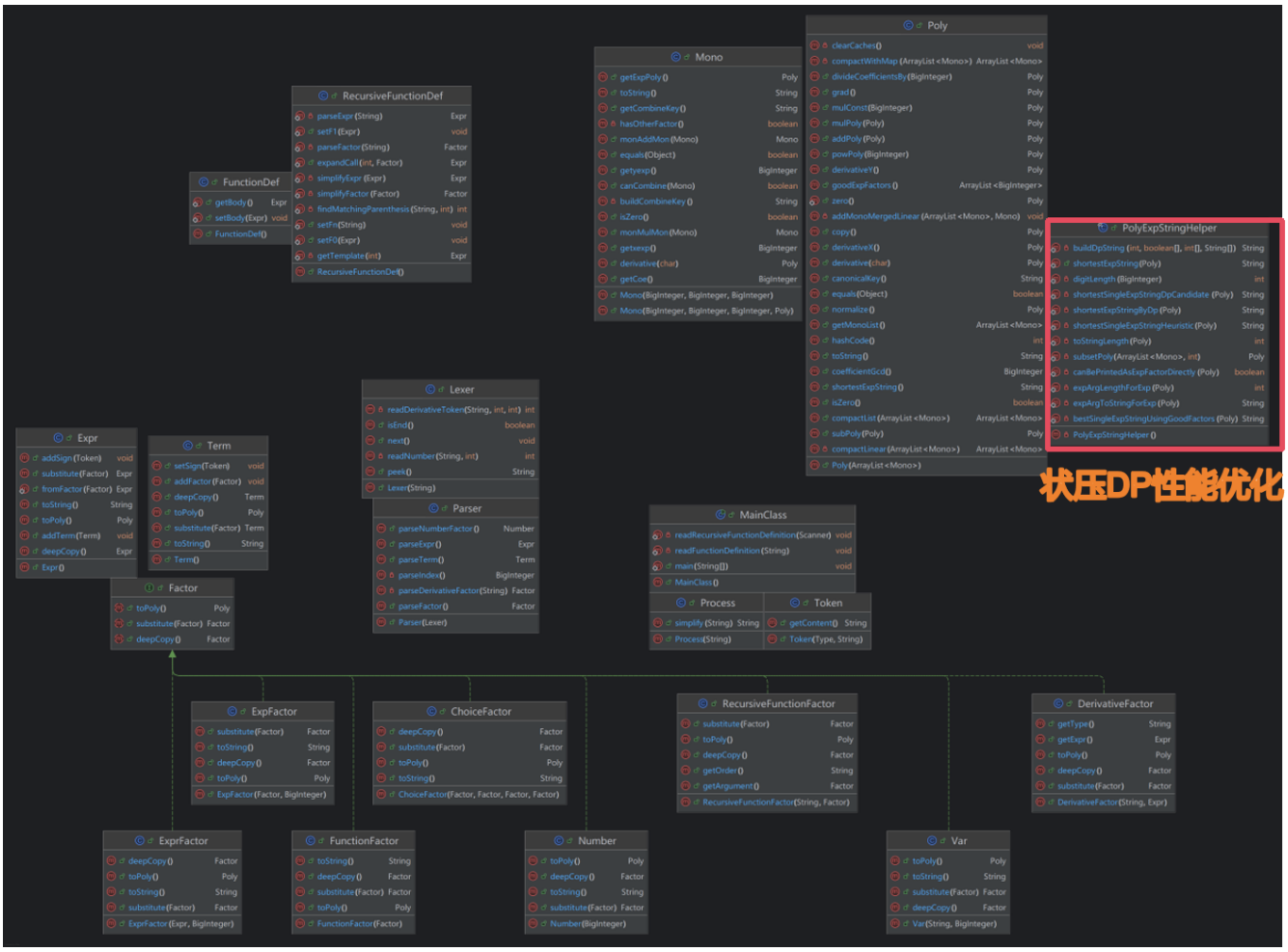
第三次作业

3.1 架构分析

本次作业在前两次的基础上，进一步新增了以下三类语言特性：

- 表达式由单变量 `x` 扩展为双变量 `x, y`
- 新增求导因子 `dx(E)`、`dy(E)` 与 `grad(E)`
- 新增递推函数 `f{n}(x)` 的定义与展开

下面给出本次作业的 UML 类图：



3.2 新功能的实现思路

3.2.1 双变量 `x`，`y` 的扩展

第三次作业中，表达式系统从单变量 `x` 扩展到了双变量 `x`，`y`。
在我的实现中，这一扩展并没有改变整体“先解析、再转标准型多项式”的主线，而是将原有单项式结构进一步推广为二元单项式。

具体来说，一个单项式 `Mono` 现在同时维护：

- 系数 `coe`
- `x` 的指数 `xexp`
- `y` 的指数 `yexp`
- 指数因子内部参数 `expPoly`

也就是说，一个单项式不再只是

$$ c \cdot x^a $$

而是可以统一表示成

$$ c \cdot x^a y^b \exp(P) $$

这样的结构。

为了支持同类项合并，`Mono` 额外维护了一个 `combineKey`，其本质由以下三部分共同决定：

- `xexp`
- `yexp`
- `expPoly` 的标准化表示

只有这三部分完全一致的单项式，才允许进行系数合并。

3.2.2 求导因子 `dx(E)`、`dy(E)` 与 `grad(E)` 的实现

`DerivativeFactor.toPoly()` 的流程是：

1. 先计算 `expr.toPoly()`，得到表达式对应的标准型多项式；
2. 若是 `dx`，调用 `inner.derivative('x')`；
3. 若是 `dy`，调用 `inner.derivative('y')`；
4. 若是 `grad`，则直接返回

$$ inner.derivative('x') + inner.derivative('y') $$

这种实现方式的优点在于：
求导逻辑被统一放在多项式层，而不是分散到每个语法树节点中，从而减少了 AST 级规则分发的复杂度。

3.2.3 `Poly` / `Mono` 层的偏导实现

真正的偏导核心在 `Mono.derivative(char var)` 中完成。

因为一个单项式被表示为

$$ c \cdot x^a y^b \exp(P) $$

所以对它求偏导时，本质上拆成两部分：

1. 对幂变量部分求导；
2. 若存在 `exp(P)`，再套用链式法则。

代码中按以下思路实现：

- 若 `xexp > 0`，先构造幂函数部分求导后的项；
- 若 `expPoly != 0`，再递归调用 `expPoly.derivative(var)`，并与原单项式相乘，补上链式法则产生的那一部分。

3.2.4 `grad(E)` 的处理方式

`grad(E)` 在数学上并不是新的独立运算，而是题目规定必须等价于：

$$ grad(E)=dx(E)+dy(E) $$

因此，我在实现中也没有为它设计新的底层运算规则，而是直接在 `DerivativeFactor.toPoly()` 中把它转成 `inner.derivative('x')`与 `inner.derivative('y')`两者求和

3.2.5 递推函数 `f{n}(x)` 的实现

$$ f_n(x)=a\cdot f_{n-1}(A)+b\cdot f_{n-2}(B)+C $$

其中：

- `a`，`b` 是常数系数；
- `A`，`B` 是递推调用时的因子参数；
- `C` 是可选附加表达式。

（1）定义阶段：保存递推模板

在读入三行定义时：

- `f{0}(x)` 保存为 `f0`
- `f{1}(x)` 保存为 `f1`
- `f{n}(x)` 则拆分出：
  - 两个递推系数 `coef1`、`coef2`
  - 两个递推参数 `arg1`、`arg2`
  - 额外项 `extra`

其中 `arg1`、`arg2` 也会先被规约和化简后再保存，方便后续统一代入。

（2）调用阶段：按模板展开并缓存

真正遇到 `f{k}(arg)` 时，程序会先生成 `f{k}(x)` 的模板表达式，然后再把真实参数 `arg` 代入进去。

这个生成过程并不是每次都从头递归，而是使用缓存数组保存已经算出的模板：

- `templates[0] = f0`
- `templates[1] = f1`
- 更高阶的 `templates[k]` 由
  - `templates[k-1]` 对 `arg1` 做代入，
  - `templates[k-2]` 对 `arg2` 做代入，
  - 再乘对应系数、加上额外项
  - 最后转成 `Poly` 化简并重新解析为 `Expr`

  得到。

因此，递推函数的展开过程本质上是：先递归生成并缓存 `f{k}(x)` 的模板，再在最外层把真实参数 `arg` 代入进去。

这样做有两个明显好处：

1. 同一个 `f{k}(x)` 只需要生成一次；
2. “生成模板”和“实际调用代入”两个步骤被清晰分离，整体结构更稳定。

3.3 表达式优化

3.3.1 已考虑到的优化

递推函数结果的缓存

递推函数的展开具有明显的重叠子问题特征。
例如在求 `f{5}(x)` 时，会反复用到 `f{4}(x)`、`f{3}(x)`、`f{2}(x)` 等结果。若每次都重新递归生成，会导致大量重复构造。

因此我将已经生成过的 `f{i}(x)` 表达式保存下来，在下次需要时直接复用。
这一点对性能的影响非常明显，也是本次作业中递推功能能够稳定运行的关键之一。

针对 `exp` 最短拆分问题的性能优化

对于指数因子，直接把相乘的 `exp` 合并虽然通常更短，但并不总是最优。因为一个 `exp(P)` 的最短形式，可能来自以下几种不同策略：

- 整体保留为 `exp(P)`；
- 提取公因数写成 `exp(Q)^k`；
- 先把 `P` 拆成若干部分，再分别优化后相乘。

如果暴力枚举所有拆分方式，复杂度会非常高。
因此我的方法是小规模精确搜索，大规模启发式处理

- 当 `exp` 参数项数较少时，用子集 DP 搜索最优拆分；
- 当项数较多时，不再做指数级搜索，而只考虑整体提取公因数的启发式方案。

先优化单块，再考虑拆分

对于任意一个指数参数块 `P`，我不会直接输出 `exp(P)`，而是先比较：

- `exp(P)`
- `exp(P/t)^t`

哪一种更短。这里的 `t` 来自系数 gcd 及其若干候选因子。

例如：

- `exp((x*2))^2` 更适合写成 `exp((x*4))`
- `exp((9*x+9*x^2))^2` 更适合写成 `exp((x^2+x))^18`

也就是说，我先在**单块内部**完成一轮“整体输出 vs 提取公因数”的局部最优选择。

对子集做 DP，寻找更优拆分

仅做整体 gcd 提取还不够，因为最优方案有时来自“先拆分，再提取”。
例如：

$$ \exp(70x^8+60x^7+\cdots+10x^2+x) $$

整体看没有最优提取方式，但若先拆成两部分，其中一部分就能提取出 10，从而得到更短输出。

因此，当项数较小时，我会把每个单项看成一个元素，对这些项做子集 DP：

- `mask` 表示一个项集；

- 先计算它整体作为一个 `exp(...)` 的最优写法；
- 再尝试把它拆成两个真子集；
- 若两部分分别优化后再相乘更短，就更新答案。

这样就把“最优拆分”转化成了“集合划分上的最短长度问题”，同时也避免了重复搜索相同子块。

因此，这套方法的本质可以概括为：

> 先把所有 `exp` 统一合并，再在输出阶段做“局部优化 + 小规模 DP + 大规模启发式”的最短表示搜索。

3.4 Bug 分析

3.4.1 强测中的问题

很荣幸在强测中获得99分，只有少许的性能上的减分~~纯狗运谢谢助教仁慈啦~~ 但是在互测中被测出了两个优化方面的Bug: 第一个是形如exp(P)类型如果需要优化为exp(S)*exp(T)且S是若干因子相乘的时候会少输出一个括号（如下图该bug发生在Poly的辅助类中）

```
if (m.getCoe().equals(BigInteger.ONE)
    && m.getxexp().equals(BigInteger.ZERO)
    && m.getyexp().equals(BigInteger.ZERO)
    && !m.getExpPoly().isZero()) {
    return !m.getExpPoly().toString().contains("*");
}
```

第二个是因为状压DP的过度优化问题导致的RTLE

3.4.2 hack

hack房间的攻击较为极其猛烈

天枢星
发起hack：1 / 42 （个人成功率：2%） 受到hack：5 / 51 （0 / 14 由你发起）

天璇星
发起hack：0 / 66 （个人成功率：0%） 受到hack：3 / 47 （0 / 14 由你发起）

天玑星
发起hack：2 / 84 （个人成功率：2%） 受到hack：6 / 44 （0 / 0 由你发起）

天权星
发起hack：10 / 36 （个人成功率：27%） 受到hack：0 / 52 （0 / 14 由你发起）

玉衡星
发起hack：4 / 42 （个人成功率：9%） 受到hack：0 / 51 （0 / 14 由你发起）

开阳星
发起hack：7 / 30 （个人成功率：23%） 受到hack：5 / 53 （2 / 14 由你发起）

摇光星
发起hack：1 / 48 （个人成功率：2%） 受到hack：6 / 50 （0 / 14 由你发起）

下面是我hack成功的输入样例：

```
1
f(x)=exp(exp(exp(exp(exp(exp(x)^2)^2)^2)^2)^2)^2
0
f(f(f(f(x))))
```

部分代码可能会因为字符串替换耗时过长而导致TLE

大模型相关使用

在第一单元的三次作业的性能优化和评测机的搭建中我都使用过AI工具（ChatGPT）。我会让 AI 给出若干优化方向，例如化简流程调整、减少中间对象生成、避免某些高复杂度操作等。AI 可以提出方向，但能否真正有效仍取决于具体代码结构，因此最后落地的优化仍需要自己筛选。 总体来说，大模型在本单元中的作用更接近一个高效率辅助工具，而不是一个可以独立完成作业的“全自动选手”。 它的优点主要是：能在卡 bug 时提供排查方向；能辅助生成测试和评测脚本。 它的不足也比较明显：对复杂逻辑的理解不一定稳定；给出的优化建议不一定真正适合当前代码；很难保证一次性精确找全 bug。 所以在我的使用过程中，AI 的主要价值是提高效率、提供思路、减少机械劳动，但最终正确性和有效性仍必须依赖人工验证。

心得体会

第一单元三次作业让我对“架构设计”和“测试”这两件事有了非常深的认识。第一次作业中，我第一次真正体会到一个灵活、包容性强的架构对于面向对象项目的重要性。好的架构不仅能够做到高内聚、低耦合，提取共性、封装功能，还能让后续增量开发时只需关注新特性本身，而不必频繁返工已有部分。感谢OO课上实验的不杀之恩，感谢OO助教在强测数据点方面的手下留情。同时也希望在后续单元中，我能和同学一起完善自动化评测工具，减少在强测和互测中的被动与狼狈，也让自己在开发过程中更加从容。

未来方向

希望课程组可以可以多出一些评测机搭建教程，方便评测机小白学习。